

第一章 绪论

1.1 数据库系统概述

1. 数据管理技术的发展

人工管理阶段 (1950 前) → 文件系统阶段 (1950-1960) → 数据库系统阶段 (1960-)

特点:

- 1. 数据不长期保存
- 2. 应用程序管理数据
- 3. 数据不共享
- 4. 数据不具有独立性

特点:

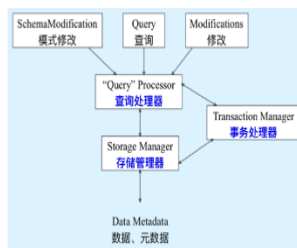
- 1. 数据可以长期保存
- 2. 由文件系统管理数据
- 3. 数据共享性差、冗余度大
- 4. 数据独立性差

特点:

- 数据结构化
- 数据共享性高、冗余度低
- 数据独立性高
- 数据由 DBMS 统一管理

2. DBMS 的主要功能: 数据定义、数据操纵、数据控制、数据组织、存储、管理
数据库的建立与维护

3. DBMS 的构成:



现实世界 → 抽象 → 设计人员完成
1.2 数据模型 { 概念模型 (信息模型) [与 DBMS 无关]
DBMS 支持的数据模型 { 逻辑模型 → 按计算机观点对数据建模, 用于 DBMS 实现
物理模型 → DBMS 完成, 描述数据在系统内部的表示方式和存取方法
三要素 { 数据结构、数据操纵、数据完整性约束 (静态) (动态)

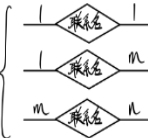
1. 联系的种类 { 1:1 一对一联系 (1:1): 实体集 A 中的任一实体, 在实体集 B 中最多有唯一实体和它对应; 反之, 实体集 B 中的任一实体, 在实体集 A 中最多有唯一实体和它对应。
1:N 一对多联系 (1:N): 实体集 A 中的任一实体, 在实体集 B 中有任意多个实体和它对应; 反之, 实体集 B 中的任一实体, 在实体集 A 中最多有唯一实体和它对应。
M:N 多对多联系 (M:N): 实体集 A 中的任一实体, 在实体集 B 中有任意多个实体和它对应; 反之, 实体集 B 中的任一实体, 在实体集 A 中有任意多个实体和它对应。

2. E-R 图

实体: 实体名

属性: 属性名

联系: 联系名



超类与子类:



3. 常用的逻辑模型

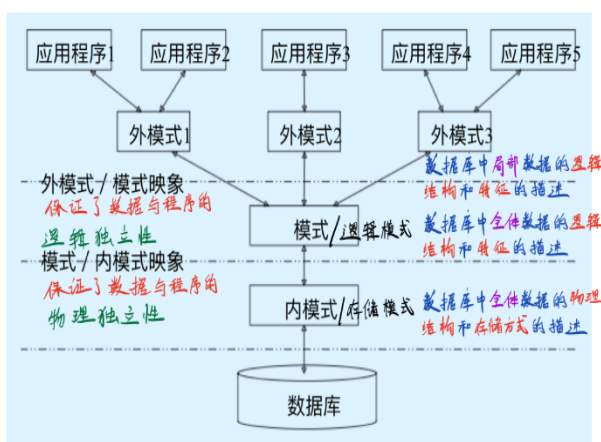
① 层次模型：只能处理1:N联系 性能优于关系模型，不低于网状模型

② 网状模型：数据结构是网

③ 关系模型：操作对象及结果均为关系(若干元组的集合)
(多对多分解为两个一对多)

优：有严格的数学推导背景
缺：查询效率较低，应优化

4. 三级模式结构



第二章 关系数据库

2.3 关系的完整性

实体完整性：主属性不能为 NULL

参照完整性：外码或者取 NULL 或者取被参照关系中某元组在该属性的取值

若FK包含Atr，
则每个Atr均取 NULL

用户定义的完整性：用户规定数据必须满足一定的要求，常用 CHECK() 约束实现

2.4 关系代数 (5种基本运算：并、差、选择、投影、笛卡尔积)

1. 关系代数是相对过程化的语言(相对于关系演算、SQL)

完整表达了为了得到结果所需进行的操作
(即先...再...)

关注查询的内容，
不太关注过程如何实现

2. 基本运算

· 选择(σ) —— 从'行'角度进行抽取

如：σ_{Sname = '张三'}(stu)

$$f_0: \pi s_{no}, s_{name} (Stu)$$

· 连接 (X)

• 等值连接 ($R \bowtie_{B=B'} S$) 变体

自然连接 ($R \bowtie S$) (在结果中去掉重复的列)

左外连接 $\bowtie$ 只保留左边关系中要合并的元组, 其余属性填 NULL

右外连接 $\bowtie$ 只保留右边关系中要舍弃的元组, 其余属性填 NULL

全外连接 $\bowtie$ 左右关系中要舍弃的元组均保留在结果中(分行), 其余属性填 NULL.

内连接即自然连接

象集 ($Z_B = b_2$): 属性组 X 上取值为 x 的诸元组在其他列上取值的集合

知:

S

B	C	D
<i>b1</i>	<i>c2</i>	<i>d1</i>
<i>b2</i>	<i>c1</i>	<i>d1</i>
<i>b2</i>	<i>c3</i>	<i>d2</i>

Cdb2 $\angle g = 120^\circ$

C	D
<i>c1</i>	<i>d1</i>
<i>c3</i>	<i>d2</i>

· 除(÷) —— 同时从“行”和“列”角度进行抽取

如:

例：A是原关系R在A'下的属性

一列行，三列属性
每行一个元组
每列一个属性
A'是A的投影
A'是R在A'下的属性

R			S		
A	B	C	B	C	D
a1	b1	c2	b1	c2	d1
a2	b3	c7	b2	c1	d1
a3	b4	c6	b2	c1	d2
a1	b2	c3			
a4	b6	c6			
a2	b2	c3			
a1	b2	c1			

R→S 结果只包含A'
“A'下的属性”
的某些取值

A

a1

3. 关系代数查询:

(9) 查询选修学号为“802”的学生所选全部课程的学生学号和姓名。 SC SC

sno	cno	grade
801	c4	92
801	c3	78
801	c2	85
802	c3	82
802	c4	90
803	c4	88

sno	cno	grade
801	c4	92
801	c3	78
801	c2	85
802	c3	82
802	c4	90
803	c4	88

$\pi_{\text{sno}, \text{sname}}(\sigma_{\text{sno} = '802'}(\pi_{\text{sno}, \text{cno}}(\sigma_{\text{sno} = '802'}(\pi_{\text{sno}, \text{cno}}(\text{SC})))))$

2.5 关系演算——元组演算

1. 写出元组演算表达式的结果表

例：

找到所有符合条件的 u, v ，然后构造出 t ，填入结果表。

R: u			S: v		
A	B	C	A	B	C
1	2	3	1	2	3
4	5	6	3	4	6
7	8	9	5	6	9

$$R5 = \{ t \mid \exists u \exists v (R(u) \wedge S(v) \wedge u[1] > v[2] \wedge t[1] = u[2] \wedge t[2] = v[3] \wedge t[3] = u[1]) \}$$

解题步骤：
1. 确定结果集的结构表，
2. 根据元组演算公式中的限定条件构造元组填入表中。

R.B	S.C	R.A
5	3	4
8	3	7
8	6	7
8	9	7

$t[1] = u[2]$
 $t[2] = v[3]$
 $t[3] = u[1]$
是构造 t 的规则。
在找到符合条件的 u, v 后即可构造出 t 。

2. 元组演算查询

例：

关系模式：

S(sno, sname, ssex, sdep, sage)

C(cno, cname, credit, cpno)

SC(sno, cno, grade) 该表属于关系(用列哪些表)

(3) 查询选修了数据库课程的学生学号、姓名和成绩。

$\{ t \mid \exists u \exists v \exists w (S(u) \wedge SC(v) \wedge C(w) \wedge u[1] = v[1] \wedge v[2] = w[1] \wedge w[2] = \text{'数据库'} \wedge t[1] = u[1] \wedge t[2] = u[2] \wedge t[3] = v[3]) \}$ 过滤、表间选择条件

(4) 查询选修了 c2 或 c4 课程的学生学号。

$\{ t \mid \exists u (SC(u) \wedge (u[2] = \text{'c2'} \vee u[2] = \text{'c4'})) \wedge t[1] = u[1] \}$ 构造结果列(3列)

关系模式：

S(sno, sname, ssex, sdep, sage)

C(cno, cname, credit, cpno)

SC(sno, cno, grade)

(5) 查询选修了 c2 和 c4 课程的学生学号。

$\{ t \mid \exists u \exists v (SC(u) \wedge SC(v) \wedge u[1] = v[1] \wedge u[2] = \text{'c2'} \wedge v[2] = \text{'c4'} \wedge t[1] = u[1]) \}$ 结果只有1列

(6) 查询不学 c2 课的学生学号

$\{ t \mid \exists u (S(u) \wedge \neg \exists v (SC(v) \wedge u[1] = v[1] \wedge v[2] = \text{'c2'})) \wedge t[1] = u[1] \}$ 不存在这样一条记录是该学生学了 c2 的

等价转换

$\{ t \mid \exists u (S(u) \wedge \neg \exists v (SC(v) \wedge u[1] = v[1] \wedge v[2] = \text{'c2'})) \wedge t[1] = u[1] \}$

注：谓词对 $\forall$ 作为蕴含式的前件，对 $\exists$ 作为合取项 一定在 u 的辖域内

(7) 查询选修了全部课程的学生学号

$\{ t \mid \exists u (SC(u) \wedge \forall v (C(v) \rightarrow \exists w (SC(w) \wedge u[1] = w[1] \wedge w[2] = v[1])) \wedge t[1] = u[1]) \}$ 对于 v 课程，都存在 u 生选了该课的记录

前束范式

$\{ t \mid \exists u \forall v \exists w (SC(u) \wedge (C(v) \rightarrow (SC(w) \wedge u[1] = w[1] \wedge w[2] = v[1])) \wedge t[1] = u[1]) \}$

(8) 查询选修学号为“802”的学生所选全部课程的学生学号

$\{ t \mid \exists u (SC(u) \wedge \forall v (SC(v) \wedge v[1] = \text{'802'} \rightarrow \exists w (SC(w) \wedge u[1] = w[1] \wedge w[2] = v[2])) \wedge t[1] = u[1]) \}$ 对于 802 所选的 v 课程，都存在 u 生选了该课的记录

关系代数，元组演算，SQL在表达能力上完全等价

第三章 SQL

3.1 SQL 概述

1. SQL = Structured Query Language 结构化查询语言

2. SQL 特点

- ① 集DQL、DDL、DML、DCL于一体
- ② 高度非过程化
- ③ 一切操作的对象及结果均为集合
- ④ 简洁易学

3.2 数据定义

1. 创建模式: create schema test authorization wang;

删除模式: drop schema test cascade/restrict

↓
连同所有 若已定义了表...
数据库对象 则拒绝删除模式
一起删除

2. 创建基本表: create table tabl

(sno char(10) not null, → 列级完整性约束
sname char(20) not null,
ssex char(2) not null,
primary key sno);

↓
表级完整性约束

能作为列级、表级约束的: primary key - foreign key - unique - check (4个)

只能作为列级约束的: default - not null (2个)

[例6] 建立一个“学生选课”表SC，它由学号Sno、课程号Cno，修课成绩Grade组成，其中(Sno, Cno)为主码。

```
CREATE TABLE SC (  
    Sno CHAR(5),  
    Cno CHAR(3),  
    Grade INT,  
    PRIMARY KEY (Sno, Cno),  
    FOREIGN KEY (Sno) REFERENCES Student(Sno),  
    FOREIGN KEY (Cno) REFERENCES Course(Cno));
```

注: ①当主码为多个属性构成的属性组时，主码只能用PRIMARY KEY (<主码属性组>)作为表级完整性约束，不必在主属性后加上NOT NULL约束，更不能加UNIQUE约束，也不能加PRIMARY KEY作为列级完整性约束；

②当定义的关系中有外码时，可以用FOREIGN KEY (<属性组>) REFERENCES <被参照表名>(<属性组>)作为表级完整性约束指定。

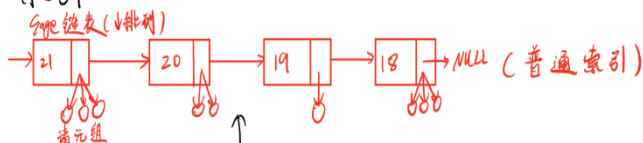
删除表: drop table tabl cascade/restrict (default)

↓ ↓
连同所有 若已定义了表...
数据库对象 则拒绝删除模式
一起删除

脏数据删除，左删右做备份，
对象一并删除，则拒绝删除

修改表: `alter table tbl` {
`add newattr int` 添加列
`alter column newattr char(10)` 修改列的数据类型
`drop newattr` 删除列
`add unique(sno)` 添加约束
`drop unique(sno)` 删除约束

3. 索引 —— 提高查询效率



分类 {
 普通索引
 单一索引: 每个索引值对应唯一的数据记录
 聚簇索引: 索引项顺序与表中数据的物理顺序一致
 (适用于很少对表进行增删操作)

建立索引

`create unique index idx1 on SC(sno, cno DESC);`
 cluster
 不写

删除索引

`drop index idx1;`

3-3 查询

1. 聚合函数: `count` 计数, `sum` 求和, `avg` 均值, `max` 最大值, `min` 最小值.

[例26] 查询学生总人数。

`SELECT COUNT(*) FROM Student;`

[例27] 查询选修了课程的学生人数。

`SELECT COUNT(DISTINCT Sno)`
`FROM SC;`

注: 用 `DISTINCT` 以避免重复计算学生人数

[例28] 查询选修1号课程的学生最高分数。

`SELECT MAX(Grade)`
`FROM SC`
`WHERE Cno = '1';`

[例29] 查询学生95002选修课程的总学分数。

`SELECT SUM(Ccredit)`
`FROM SC, Course`
`WHERE Sno = '95002' AND SC.Cno = Course.Cno;`

2. 分组输出

SELECT子句: 指定要显示的属性列
FROM子句: 指定查询对象(基本表或视图)
WHERE子句: 指定查询条件
GROUP BY子句: 对查询结果按指定列的值分组
HAVING短语: 筛选出满足指定条件的组
ORDER BY子句: 对查询结果按指定列的值升序或降序排序

使用GROUP BY子句后, SELECT子句的列名列表中只
 能出现分组属性和聚合函数

[例31] 查询选修了多于3门课程的学生学号。

```

SELECT Sno
FROM SC
GROUP BY Sno HAVING COUNT(*) > 3;

```

→ 作用于各组上

[例32] 查询有多于3门课程是90分以上的学生的学号及(90分以上的)课程数。

```

SELECT Sno, COUNT(*)
FROM SC
WHERE Grade >= 90
GROUP BY Sno HAVING COUNT(*) > 3;

```

→ 先选出所有 grade > 90 的元组
再分组 - 统计

3. 连接查询

• 自身连接

[例34] 查询每一门课的间接先修课 (即先修课的先修课)。

```

SELECT FIRST.Cno, SECOND.Cpno
FROM Course FIRST, Course SECOND
WHERE FIRST.Cpno = SECOND.Cno;

```

注:

- (1) 一个表与其自己进行连接, 称为表的**自身连接**
- (2) 需要**给表起别名**以示区别
- (3) 由于所有属性名都是同名属性, 因此必须使用别名前缀

- 左外连接 `select ... from student left outer join SC;`
- 右外连接 `select ... from student right outer join SC;`
- 内连接 `select ... from student inner join SC;`
(自然连接)

4. 嵌套查询

相关子查询: 子查询的查询条件与父查询相关

不相关子查询: 子查询的条件与父查询当前值无关

不相关子查询一定可用连接查询完成,

相关子查询不一定可用连接查询完成

[例40] 找出每个学生超过他选修课程平均成绩的课程号。

```
SELECT Sno, Cno
FROM SC X
WHERE Grade >= ( SELECT AVG(Grade)
                  FROM SC Y
                  WHERE Y.Sno = X.Sno);
```

↓ 先算出这个

此查询的执行过程：

- (1) 首先取外层查询中表的第一个元组，将其Sno的值传递给内层查询；
- (2) 执行内层查询，根据结果再执行外层查询；
- (3) 取外层表的下一个元组，重复这一过程，直至外层表全部检查完为止。

这种查询称为**相关子查询**，即子查询的条件与父查询当前值相关。

谓词：ANY：某些，ALL：所有

如：select ... from ... where grade >= ANY (select grade from ... where ...)

注：用集函数实现子查询的效率高于用ALL/ANY。

5. 带exists谓词的子查询

注：(1) 多数带EXISTS或NOT EXISTS谓词的子查询能够被其他形式的子查询等价替换。

(2) 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换。

例子：

[例45] 查询选修了全部课程的学生姓名。

元组演算表达式：

$$\{t | \exists u(S(u) \wedge \forall v(C(v) \rightarrow \exists w(SC(w) \wedge w[1]=u[1] \wedge w[2]=v[1])) \wedge t[1]=u[2])\}$$

连接SC和S 连接SC和C

对于C中任意一个课程，都存在他

选课课元等价转换

$$\{t | \exists u(S(u) \wedge \neg \exists v(C(v) \wedge \neg \exists w(SC(w) \wedge w[1]=u[1] \wedge w[2]=v[1])) \wedge t[1]=u[2])\}$$

最后选出所需

利用否定后在子查询中

自然语义：输出这样的学生，不存在某门课程，在他的选课记录里没有选这门课。

* SQL查询语句：

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
  ( SELECT
    FROM Course
    WHERE NOT EXISTS
      ( SELECT
        FROM SC
        WHERE Sno = Student.Sno
          AND Cno = Course.Cno );
```

连接SC和Student

连接SC和Course

[例46] 查询至少选修学生95002选修的全部课程的学生学号。

元组演算表达式：对于任意一门95002选修的课程，都存在该生选修

$\{t | \exists u(SC(u) \wedge \forall v(SC(v) \wedge v[1]='95002' \rightarrow \exists w(SC(w) \wedge w[1]=u[1] \wedge w[2]=v[2])) \wedge t[1]=u[1])\}$

⇕ 等价转换

$\{t | \exists u(SC(u) \wedge \neg \exists v(SC(v) \wedge v[1]='95002' \wedge \neg \exists w(SC(w) \wedge w[1]=u[1] \wedge w[2]=v[2])) \wedge t[1]=u[1])\}$

对于该生而言，不存在这样的课程，95002选了但该生没选

自然语义：输出这样的学生，不存在'95002'选了的课程，在他的选课记录中没有出现。

* SQL查询语句：

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS
(
    SELECT *
    FROM SC SCY
    WHERE SCY.Sno = '95002' AND NOT EXISTS
    (
        SELECT *
        FROM SC SCZ
        WHERE SCZ.Sno = SCX.Sno AND
        SCZ.Cno = SCY(Cno);
    )
)
```

必须起别名，便于写最后的where连接条件

6. 集合查询 —— 将两个SELECT查询块用集合操作语句联结起来

- 并 (UNION)
- 交 (INTERSECT)
- 差 (EXCEPT)

参与操作的结果表列数及各列数据类型应相同

7. 一些查询的注意事项

✱ 集函数只能用于SELECT子句和HAVING短语之中，而绝对不能直接出现在WHERE子句中。如 `select count(cno) from ... having count(*) > 3 ...`

✱ 使用了分组的查询语句，其SELECT子句中只能出现分组属性和集函数，而不能有在GROUP BY没有出现的属性。

原因在于SELECT子句是对分组后的结果集进行输出，参与分组的属性在同一组中取相同值，而不参与分组的属性(不在GROUP BY子句中出现的属性)则可能取不同的值，分组输出通常是对组的整体描述，而不是组内的详细记录，不参与分组的属性取值不同，故无法输出。

✱ **ORDER BY**子句用于对查询结果进行排序后再输出，故只用于**最外层**的查询，而子查询中不应该出现**ORDER BY**子句。

✱ 子查询的结果集可以看作基本关系一样作为查询的数据源。此时必须**对子查询命名别名**，否则为语法错误。

例：查询全体学生的学号、姓名和平均成绩。

```
SELECT Student.Sno, Sname, avg_grade
FROM Student, (SELECT Sno, AVG(Grade) AS avg_grade
                FROM SC
                GROUP BY Sno) AS SC_AVG
WHERE Student.Sno = SC_AVG.Sno;
```

3.4 数据更新

1. 插入数据：
$$\begin{cases} \text{insert into tabl values (...);} \\ \text{insert into tabl(sno, sname) values ('001', '张三');} \end{cases}$$
2. 修改数据：
$$\begin{array}{l} \text{update student} \\ \text{set sage = 20} \\ \text{where sno = '0001'; (单个)} \end{array} \quad \begin{array}{l} \text{update student} \\ \text{set sage = sage + 1} \\ \text{where sdept = 'CS'; (全体)} \end{array}$$
3. 删除数据：
$$\begin{array}{l} \text{delete from student} \\ \text{(删元组) where sno = '0001'; (单个)} \end{array} \quad \begin{array}{l} \text{delete from SC (删除所有元组)} \end{array}$$

3.5 视图

1. 视图对应外模式

2. [例2] 建立信息系学生的视图，并要求透过该视图进行的更新操作只涉及信息系学生。

```
CREATE VIEW IS_Student2
AS
SELECT *
FROM Student
WHERE Sdept='IS'
WITH CHECK OPTION;
```

针对此视图的更新，更新后每一行都应满足Sdept='IS'条件

插入操作：不能插入('001', '张', '女', 19, 'MA')

修改操作：不能将Sdept字段改为'MA'

删除操作：无影响

3. 删除视图

drop view view-1

第四章 数据库安全性

4.2.1 用户标识与识别

4.2.2 自主存取控制(DAC) —— 授权与回收

• GRANT ... TO ... 和 REVOKE ... FROM ...

[例1] 把查询Student表权限授给用户U1。

```
GRANT SELECT
ON TABLE Student
TO U1;
```

[例2] 把对Student表和Course表的全部权限授予用户U2和U3。

```
GRANT ALL PRIVILEGES
ON TABLE Student, Course
TO U2, U3;
```

[例3] 把对表SC的查询权限授予所有用户

```
GRANT SELECT
ON TABLE SC
TO PUBLIC; 关键字
```

[例4] 把查询Student表和修改学生学号的权限授给用户U4。

```
GRANT UPDATE(Sno), SELECT
ON TABLE Student
TO U4;
```

收回权限——从指定用户那里收回对指定对象的指定权限。

使用REVOKE语句:

```
REVOKE <权限>[, <权限>]...
ON <对象类型> <对象名>[, <对象类型> <对象名>]...
FROM <用户>[, <用户>]... [CASCADE|RESTRICT];
```

[例6] 把用户U4修改学生学号的权限收回。

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4; CASCADE (级联收回)
```

[例7] 收回所有用户对表SC的查询权限。

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```

• 角色——一组权限的集合(1表一类用户)
可能化授权过程

1. 首先创建一个角色 R1

```
CREATE ROLE R1;
```

2. 然后使用GRANT语句, 使角色R1拥有Student表的SELECT, UPDATE, INSERT权限

```
GRANT SELECT, UPDATE, INSERT
ON TABLE Student TO R1;
```

3. 将这个角色授予王平, 张明, 赵玲。使他们具有角色R1所包含的全部权限

```
GRANT R1 TO 王平, 张明, 赵玲;
```

4. 可以一次性通过R1来回收王平的这3个权限

```
REVOKE R1 FROM 王平;
```

4.2.3 强制存取控制(MAC)

1. 主体与客体

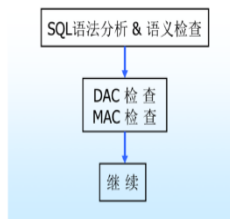
- 主体是系统中的活动实体
 - DBMS所管理的实际用户
 - 代表用户的各进程
- 客体是系统中的被动实体，是受主体操纵的
包括：文件、基表、索引、视图

2. 敏感度标记：
 $\left. \begin{array}{l} \text{绝密} > \text{机密} > \text{秘密} \\ > \text{公开} \end{array} \right\} \begin{array}{l} \text{主体} \text{ --- 许可证级别} \\ \text{客体} \text{ --- 密级} \end{array}$

3. MAC 规则

- (1) 仅当主体的许可证级别 **大于或等于** 客体的密级时，该主体才能 **读取** 相应的客体；
- (2) 仅当主体的许可证级别 **等于** 客体的密级时，该主体才能 **写** 相应的客体。 **防止高级别主体泄密**

4. * DAC + MAC 安全检查示意图



4.2.4 视图机制

[例13] 在学籍管理数据库中，每个学生有以自己的学号为用户名和密码的帐号，要求每个学生登录后只能访问自己的信息

方法一：授予权限时使用存取谓词；

GRANT SELECT ON Student WHEN USER()=Sno TO ALL;

方法二：使用视图(SQL Server2000为例)：

(1) DBA建立一个视图，将视图的访问权限授予所有用户；

CREATE VIEW My_Student

AS

SELECT * FROM Student WHERE Sno=USER--系统变量

GRANT SELECT ON My_Student TO PUBLIC;

(2) 学生使用自己的用户名登录

SELECT * FROM Student;--无权访问

SELECT * FROM My_Student;--只得到自己的信息

4.2.5 审计 —— 预防、监测

将用户对DB的操作记录在审计日志上，DBA可根据日志检测到非法存取数据者。

4.2.6 数据加密

4.3 统计数据库安全性

允许查询聚合函数信息 (sum, avg, ...)
不允许查询基表记录信息

规则1：任何查询至少要涉及N(N足够大)个以上的记录

规则2：任意两个查询的相交数据项不能超过M个

规则3：任一用户的查询次数不能超过 $1+(N-2)/M$

第五章 数据库完整性

防止出现不合语义的、不正确的数据

1. 参照完整性

FOREIGN KEY (Sno) REFERENCES Student(Sno)

ON DELETE CASCADE 级联删除

ON UPDATE NO ACTION, 当更新 Student 表中的 Sno 造成与 SC 表不一致时
拒绝更新。

2. 用户定义的完整性

[例5] 建立学生登记表 Student, 要求学号在 90000~99999 之间, 姓名不能取空值, 年龄小于 30, 性别只能是“男”或“女”。

```
CREATE TABLE Student
(Sno NUMERIC(6)
CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999),
Sname CHAR(20)
CONSTRAINT C2 NOT NULL,
Sage NUMERIC(3)
CONSTRAINT C3 CHECK (Sage < 30),
Ssex CHAR(2)
CONSTRAINT C4 CHECK (Ssex IN ('男', '女')),
CONSTRAINT StudentKey PRIMARY KEY(Sno)
);
```

[例6] 修改表 Student 中的约束条件, 要求学号改为在 900000~999999 之间, 年龄由小于 30 改为小于 40。

思路: 可以先删除原来的约束条件, 再增加新的约束条件

ALTER TABLE Student DROP CONSTRAINT C1;

```
ALTER TABLE Student
ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000
AND 999999);
```

ALTER TABLE Student DROP CONSTRAINT C3;

```
ALTER TABLE Student
ADD CONSTRAINT C3 CHECK (Sage < 40);
```

3. 域

[例7] 建立一个性别域, 并声明性别域的取值范围。

```
CREATE DOMAIN GenderDomain CHAR(2)
```

```
CONSTRAINT GD CHECK (VALUE IN ('男', '女'));
```

定义 Student 时可以声明 Ssex 属于 GenderDomain。(相当于 GenderDomain 是一个数据类型)

4. 触发器

```
CREATE TRIGGER <触发器名>
{BEFORE | AFTER} <触发事件> ON <表名>
FOR EACH {ROW | STATEMENT}
[WHEN <触发条件>]
<触发动作体>
```

注:

- 触发器可以在触发事件之前或之后执行;
- 触发事件为 INSERT/DELETE/UPDATE;
- 按照触发动作分为行级触发器和语句级触发器;
- 当触发器被激活时, 触发条件为真才执行触发动作体;
- 同一个表上的多个触发器遵循如下的执行顺序: 执行该表上的 BEFORE 触发器; 激活触发器的 SQL 语句; 执行该表上的 AFTER 触发器。
- 触发器不属于 SQL 规范, 不同的 DBMS 语法不同;

第六章 关系数据库理论

6.1 关系模式设计引论

· 存储异常 —— 根源：属性间的依赖关系

更新异常 (Update Anomalies)

数据冗余，更新数据时，维护数据完整性代价大。

例：某系更换系主任后，系统必须修改与该系学生有关的每一个元组

插入异常 (Insertion Anomalies)

该插的数据无法插入到表中。

例：如果一个系刚成立，尚无学生，我们就无法把这个系及其系主任的信息存入数据库。

删除异常 (Deletion Anomalies)

不该删除的数据不得不删。

例：如果某个系的学生全部毕业了，在删除该系学生信息的同时，把这个系及其系主任的信息也丢掉了。

结论：

“好”的模式：不会发生插入/删除/更新异常且数据冗余量少。

6.2 规范化

规范化总结：

关系数据库的规范化理论是数据库逻辑设计的工具；

一个关系只要其分量都是不可分的数据项，它就是规范化的关系，但这只是最基本的规范化(1NF)；

规范化程度有多个不同的级别；

规范化程度较低的关系不一定能够很好地描述现实世界，存在插入异常、删除异常、修改复杂、数据冗余等问题；

一个较低级别范式的关系模式，可以通过模式分解转换为若干个较高级别范式的关系模式集合；

3NF $\rightarrow$ BCNF：消除了主属性对不包含它的码的PFD、TFD。

BCNF判定：所有非平凡的FD的决定因素包含码

若 $Z = U - X - Y$, $X \rightarrow Y$, 则：

- ① 若 Y 或 Z 范围扩大，则4NF不一定成立
- ② 若 Y 不变， Z 缩小，则4NF仍成立。

6.3 数据依赖的公理系统

1. Armstrong 公理系统的三条基本推理规则

PS: 实用推理规则

- ① 自反规则: 若 $X \rightarrow Y$, $X \subseteq Y$, 则 $X \rightarrow Y$
- ② 分解规则: 若 $X \rightarrow Y$, $Z \subseteq Y$, 则 $X \rightarrow Z$
- ③ 传递规则: 若 $X \rightarrow Y$, $Y \rightarrow Z$, 则 $X \rightarrow Z$

自反律：若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 为F所蕴含；增广律：若 $X \rightarrow Y$, $Z \subseteq U$, 则 $XZ \rightarrow YZ$ 为F所蕴含。

传递律：若 $X, Y, Z \subseteq U$, $X \rightarrow Y$, $Y \rightarrow Z$, 则 $X \rightarrow Z$ 为F所蕴含

2. 函数依赖集的闭包 F^+ 的计算是NP完全问题

可给出F能推出的全部FD, 但信息庞大, 难以利用

属性集的闭包 X^+ (求解简单, 肉眼可观察出结果)

3. 极小函数依赖集 (最小依赖集、最小覆盖) 注: 极小FD集可能不唯一。

制定条件: ① 所有FD左部仅含一个属性

② 左部无冗余属性

③ 无多余的FD

求解方法：① 分解右部属性
② 去除左部的冗余属性
③ 去除多余的FD

例：

例：设关系模式R(ABCDE)上的函数依赖集F={A→BC, BCD→E, B→D, A→D, E→A}，求F的极小函数依赖集。

解：① 对每个函数依赖分解右部属性，得：

$F_1 = \{A \rightarrow B, A \rightarrow C, BCD \rightarrow E, B \rightarrow D, A \rightarrow D, E \rightarrow A\}$

② 去除左部冗余属性 *你存在*

∵ (BC)→E，包含E，BCD→E中的D为冗余属性，以BC→E取代BCD→E，得：

$F_2 = \{A \rightarrow B, A \rightarrow C, BC \rightarrow E, B \rightarrow D, A \rightarrow D, E \rightarrow A\}$

③ 去除多余函数依赖

∵ A→B, B→D可以推导出A→D，故A→D多余，去

4. 候选码求解算法

首先，求出L、R、LR、N

对于给定的关系模式R<U, F>，依照函数依赖集F将U中的属性分为以下四类：

L类属性：在F中只出现在函数依赖的左部的属性；

R类属性：在F中只出现在函数依赖的右部的属性；

LR类属性：分别出现在F中的函数依赖左部和右部的属性；

N类属性：不在F中的函数依赖中出现的属性。

结论：

① L类属性和N类属性**必包含于**任何候选码中；

② R类属性**必不包含于**任何候选码中；

③ LR类属性**不能确定**是否在候选码中。

令 $X = L \cup N, Y = LR$

然后进行正式求解：

① 若 $X_F^+ = U$ ，则X为唯一候选码，算法结束；

否则：

② 逐一取Y中的单一属性A与X并起来，若 $(XA)_F^+ = U$ ，则XA为候选码

令 $Y = Y - A$ 。若此时 $Y = \emptyset$ ，算法结束；

否则：

③ 依次取Y中的任意2/3/...个属性Z与X并起来，若 (XZ) 不包含已求得的候选码且 $(XZ)_F^+ = U$ ，则XZ为候选码。重复上述操作直到取到的 $Z = Y$ 为止，算法结束。

例：设 $R<U, F>$ ，其中：
 $U=ABCDE$, $F=\{A\rightarrow BC, CD\rightarrow E, B\rightarrow D, E\rightarrow A\}$ ，求R的所有候选码。

解：(1) R中无L、N类属性，A, B, C, D, E均为LR类属性，故 $X=\Phi$, $Y=ABCDE$ ；
 (2) $X_F^+=\Phi \neq U$ ，继续；

(3)取A，则 $A_F^+=ABCDE=U$ ，
 A为候选码；更新 $Y=BCDE$
 取B、C、D，其闭包均不等于全属性集U；
 取E，则 $E_F^+=ABCDE=U$ ，
 E为候选码；更新 $Y=BCD$
 $Y=BCD$

(4)在Y中任取两个属性判定
 $(BC)_F^+=BCDEA=U$ ，BC为候选码；(不改变Y)
 $(BD)_F^+$ 不等于全属性集U，BD不是候选码；
 $(CD)_F^+=CDEAB=U$ ，CD为候选码；

BCD包含已求得的候选码BC，
 BCD不是码，算法结束。
 故关系R的所有候选码为：A，E，BC，CD。

6.4 模式分解

规范化过程就是对关系模式进行分解，使其达到更高的范式级别，以避免或减少更新异常和数据冗余。
 若R中仅有FD，则R4NF最佳，若R中有MVD，则4NF最佳。

1. 模式分解的定义

关系模式 $R<U, F>$ 的一个分解是指 $P=\{R_1<U_1, F_1>, R_2<U_2, F_2>, \dots, R_n<U_n, F_n>\}$
 其中 $U=\bigcup_{i=1}^n U_i$ ，且没有 $U_i \subseteq U_j, 1 \leq i, j \leq n$ ； F_i 是F在 U_i 上的投影。
 如： $U=ABCDE$ ， $U_1=AB$ ， $U_2=BCE$ ， $U_3=ABDE$ ，则由于 $U_1 \subseteq U_3$ ，故这不是一个分解

2. 模式分解应考虑的问题

- ① 分解不能丢失信息——无损连接性
- ② 分解应尽量避免更新异常、数据冗余——保持函数依赖性

3. 无损分解(分解保持无损连接性)：分解后的表进行自然连接=原表
 注：无损连接性和分解后的 $F_1, F_2, \dots$ 无关

4. 无损连接性的判定

• 一分为二的分解

关系模式 $R<U, F>$ 的一个分解 $P=\{R_1<U_1, F_1>, R_2<U_2, F_2>\}$ 具有无损连接性的必要条件是

$U_1 \cap U_2 \rightarrow U_1 - U_2$ 或 $U_1 \cap U_2 \rightarrow U_2 - U_1$ 成立 (是否成立通过看 $(U_1 \cap U_2)_F^+$ 是否包含 $U_1 - U_2$ 判断)

· 一分为多的分解

(1) 建立一张 n 列 k 行的表，每一列对应一个属性 A_j ，每一行对应一个关系模式 R_i 。

若属性 A_j 属于 U_i ，则在表中 i 行 j 列处填上 a_j ，否则填上 b_{ij} ；

(2) 把表看作 $R<U, F>$ 的一个关系，依次检查 F 中每一函数依赖在表中是否成立。

若不成立，则做出修改使其成立。修改原则：能改成 a 则改成 a ，否则改成行下标最小的 b (称为一趟Chase过程)；

(3) 如果在一趟Chase中的某次更改之后出现形如 $a_1, a_2, \dots, a_n$ 的行，算法结束，分解 ρ 具有无损连接性；否则，继续下一趟Chase过程，直到一趟Chase之后表无任何变化时算法结束(此时未出现形如 $a_1, a_2, \dots, a_n$ 的行)，分解 ρ 不具有无损连接性。

例1:

例：已知 $R<U, F>$ ， $U=ABCDE$ ， $F=\{AB \rightarrow C, D \rightarrow E, C \rightarrow D\}$ ， R 的一个分解 $\rho = \{R_1(ABC), R_2(CD), R_3(DE)\}$ 。判定分解 ρ 是否为无损连接的分解。

解：(1) 构造初始表：

$R_i \backslash U$	A	B	C	D	E
(ABC)	a_1	a_2	a_3	b_{14}	b_{15}
(CD)	b_{21}	b_{22}	a_3	a_4	b_{25}
(DE)	b_{31}	b_{32}	b_{33}	a_4	a_5

(例)：....., $F=\{AB \rightarrow C, D \rightarrow E, C \rightarrow D\}$,

解：(1) 构造初始表：

$R_i \backslash U$	A	B	C	D	E
(ABC)	a_1	a_2	a_3	b_{14}	b_{15}
(CD)	b_{21}	b_{22}	a_3	a_4	b_{25}
(DE)	b_{31}	b_{32}	b_{33}	a_4	a_5

(2) 第一趟Chase:

① $AB \rightarrow C$: 无 AB 列上取值相同的行，不做修改；

② $D \rightarrow E$: 第二、三行在 D 上的取值均为 a_4 ，第三行 E 列出现 a_5 ，将这两行在 E 上的取值修改为 a_5 ；

(例)：....., $F=\{AB \rightarrow C, D \rightarrow E, C \rightarrow D\}$,

(2) 第一趟Chase:

①

②

$R_i \backslash U$	A	B	C	D	E
(ABC)	a_1	a_2	a_3	b_{14}	b_{15}
(CD)	b_{21}	b_{22}	a_3	a_4	a_5
(DE)	b_{31}	b_{32}	b_{33}	a_4	a_5

③ $C \rightarrow D$: 第一、二行在 C 上的取值为 a_3 ，将这两行在 D 上的值改为 a_4 ；

(例)：....., $F=\{AB \rightarrow C, D \rightarrow E, C \rightarrow D\}$,

(2) 第一趟Chase后的结果:

$R_i \backslash U$	A	B	C	D	E
(ABC)	a_1	a_2	a_3	a_4	b_{15}
(CD)	b_{21}	b_{22}	a_3	a_4	a_5
(DE)	b_{31}	b_{32}	b_{33}	a_4	a_5

(3) 第二趟Chase:

① $AB \rightarrow C$: 无改变；

② $D \rightarrow E$: 第一、二、三行在 D 上的取值为 a_4 ，将它们 E 上的取值改为 a_5 ；

(4) 表第一行出现 a_1, a_2, a_3, a_4, a_5 ，故分解 ρ 为无损分解。

例2: 见另一PDF.

5. 保持函数依赖性的判定

(函数依赖集的投影算法)

例1:

设关系模式 $R(ABCDE, F)$ 上的函数依赖集 $F=\{A \rightarrow BC, BCD \rightarrow E, B \rightarrow D, A \rightarrow D, E \rightarrow A\}$, $\rho=\{R_1(ABD), R_2(ACE)\}$ 是关系模式 R 的一个分解。求 F_1, F_2

$$U_1 = ABD$$

$$U_2 = ACE$$

解: 先求 F_1 :

单属性:

$$(A)_F^+ = ABCDE = U, (B)_F^+ = BD, (D)_F^+ = D$$

$\Rightarrow$ 更新 $F_1 = \{A \rightarrow B, A \rightarrow D, B \rightarrow D\}$ (即对于每个属性集的闭包, 都要在 F_1 中添加能由该闭包得到的 FD)

$$(BD)_F^+ = BD$$

$\Rightarrow F_1$ 不变

三属性: 由于 ABD 包含 A , 故无需求其闭包。

$\therefore F_1$ 最终为 $\{A \rightarrow B, A \rightarrow D, B \rightarrow D\}$ eg: $BC \rightarrow A, B \rightarrow A$, 则应去除 $BC \rightarrow A$ (C 是冗余属性)

再消除 F_1 中的左部冗余属性 (本例无) 和多余的 FD (若某 FD 能由其他 FD 推出, 则应去除)

由于 $A \rightarrow B, B \rightarrow D$ 能推出 $A \rightarrow D$, 故 $A \rightarrow D$ 多余, 去除。

综上, $F_1 = \{A \rightarrow B, B \rightarrow D\}$

再求 F_2 :

单属性:

$$(A)_F^+ = ABCDE = U, (C)_F^+ = C, (E)_F^+ = AEBCD = U$$

$$\Rightarrow \text{更新 } F_2 = \{A \rightarrow C, A \rightarrow E, E \rightarrow A, E \rightarrow C\}$$

双属性: 不能包含 A 或 E , 则无符合条件的双属性

故也无符合条件的三属性

$\therefore F_2$ 最终为 $\{A \rightarrow C, A \rightarrow E, E \rightarrow A, E \rightarrow C\}$

再消除 F_2 中的左部冗余属性 (本例无) 和多余的 FD

由于 $E \rightarrow A, A \rightarrow C$ 能推出 $E \rightarrow C$, 故 $E \rightarrow C$ 冗余, 删去。

综上, $F_2 = \{A \rightarrow C, A \rightarrow E, E \rightarrow A\}$

下面判断分解 ρ 是否保持函数依赖:

$F_1 \cup F_2 = \{A \rightarrow BCE, B \rightarrow D, E \rightarrow A\}$ (这里 FD 右部合并了属性, 便于观察)

然后求 F 中每个 FD 左部在 $F_1 \cup F_2$ 上的闭包。

若包含该 FD 左部, 则说明该功能由 F_1, F_2 推出 (即保持了该 FD)
否则说明分解后未保持该 FD。

$$\textcircled{1} (A)_{F_1 \cup F_2}^+ = ABCDE \supseteq BC \Rightarrow \text{分解保持了 } A \rightarrow BC$$

$$\textcircled{2} (BCD)_{F_1 \cup F_2}^+ = BCD \not\supseteq E \Rightarrow \text{分解未保持 } BCD \rightarrow E$$

综上, 分解 ρ 未保持函数依赖。

有关模式分解的结论

- (1) 无损连接性能在分解为4NF的过程中实现, 即关系模式可无损连接地分解为4NF;
- (2) 关系模式可以在保持函数依赖性的情况下分解为3NF, 未必能达到BCNF;
- (3) 既保证无损连接性, 又保持函数依赖的分解可以达到3NF, 未必能达到BCNF。

小结

依照函数依赖和多值依赖将关系模式分为1NF~4NF, 用来判定更新异常和数据冗余; 判断范式需求出所有候选码; 数据依赖的公理系统给出了一组数据依赖的推理规则, 由此可以推导出属性(组)之间的数据依赖; 函数依赖集的闭包 F^+ 是该关系模式上所有的函数依赖的全集; 属性集的闭包 XF^+ 是所有能由X决定的属性的集合; 极小函数依赖集 F_{min} 是等价于F的最精简的函数依赖集; 模式分解的目的是为了使关系模式达到更高的范式, 以减少或消除更新异常和数据冗余, 在函数依赖的范畴内模式应分解至3NF, 最好到BCNF, 含有多值依赖则最好至4NF; 模式分解的评判准则是无损连接性和保持函数依赖性, 分解必须保证无损连接性而尽量保持函数依赖性。

0

第七章 数据库设计

1、数据库应用设计流程: 需求分析 → 概念设计 → 逻辑设计 → 物理设计
数据流图... ER图... 设计关系模式、数据库表视图... 选择存取方法(索引、聚簇、HASH) 确定存储结构

第八章 数据库编程

1、SQL与主语言如何通信?

1. SQL通信区(SQLCA, SQL Communication Area)

SQL语句执行后,描述系统工作状态和运行环境的数据区:

- (1) 向主语言传递SQL语句的执行状态信息(如成功/失败);
- (2) 主语言能够据此控制程序流程;

2. 主变量

SQL语句中使用的主语言程序变量,分为:

- (1) 输入主变量:主语言向SQL语句提供参数;
- (2) 输出主变量:SQL语句的查询结果返回给主语言;

3. 游标

存放SQL语句执行结果的数据缓冲区,通过循环逐一获取记录,赋给主变量,交由主语言进一步处理。

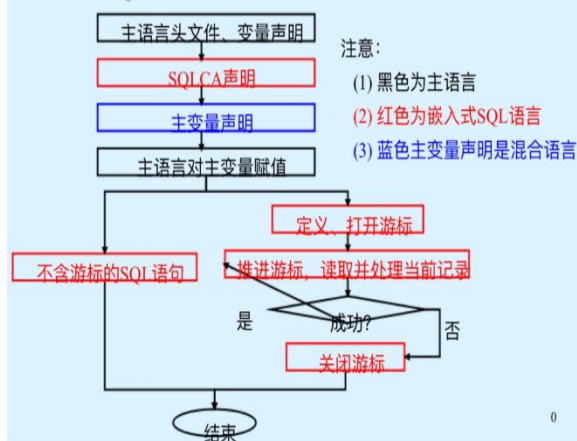
2.

使用游标的步骤:

- (1) 说明游标;
- (2) 打开游标,把所有满足查询条件的记录取至缓冲区;
- (3) 推进游标指针,把当前记录从缓冲区中取出来送至主变量;
- (4) 检查当前记录是否是要修改或删除的记录,是则进行处理;
- (5) 重复第(3)和(4)步,逐条取出结果集中的记录进行判断和处理,一直到处理完毕;
- (6) 关闭游标,释放结果集占用的缓冲区和其他资源。

3.

嵌入式SQL的程序流程:



第十章 数据库恢复技术

1. 事务：用户定义的一个操作序列，这些操作要么都做，要么都不做。

事务是恢复和并发控制的基本单位

2. 定义事务：
BEGIN TRANSACTION
SQL 语句
:
COMMIT/ROLLBACK

(1) COMMIT: 提交事务对数据库的所有更新操作，事务执行正常结束；

(2) ROLLBACK: 事务异常终止，发生了故障不能继续执行，回滚事务的所有更新操作，回到事务开始时的状态；

3. 事务的特性：A、C、I、D

原子性(Atomicity)

事务中包括的所有操作要么都做，要么都不做。

一致性(Consistency)

事务执行结果必须使数据库从一个一致性状态变到另一个一致性状态。

隔离性(Isolation)

一个事务的执行不能被其他事务干扰，从而影响它对数据的正确使用和修改。

持续性(Durability)

一个事务一旦提交，它对数据库的改变就应该是永久的，接下来的其他操作或故障不应该对其执行结果有任何影响。

4. 故障及恢复策略

• 事务故障：事务在运行至终点前中止(夭折)

恢复方法：撤消(Undo)事务对数据库的修改

恢复步骤：

- (1) 反向扫描日志文件(从后开始)，查找该事务的更新操作；
- (2) 对更新操作执行逆操作，将“更新前的值”写入数据库：对插入执行删除，对删除执行插入，对修改执行修改；
- (3) 继续反向扫描，查找下一个更新操作，执行(2)；
- (4) 扫描到该事务的开始标记，事务故障恢复结束。

• 系统故障：
(1) 未完成事务对数据库的更新已写入数据库(本不应在这时写入)
(2) 已提交事务对数据库的更新留在缓冲区，未写入数据库(本应写入)

恢复方法：撤消(Undo)未完成的事务 + 重做(Redo)已完成的事务

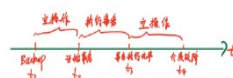
恢复步骤：

- (1) 正向扫描日志文件，得到Redo队列与Undo队列；
- (2) 对Undo队列事务进行Undo处理(反向扫描)；
- (3) 对Redo队列事务进行Redo处理(正向扫描)。

• 介质故障：

恢复方法：重装数据库，恢复到一致性状态，Redo已完成的事务

恢复步骤：装入最新的Backup后备副本 + 装入日志文件副本，Redo事务



第十一章 并发控制

1. 并发操作可能出现的问题 (可能破坏事务的隔离性与一致性)

- 丢失修改
- 不可重复读
- 读“脏”数据

T1	T2
① 读A=16	
②	读A=16
③ $A \leftarrow A-1$ 写回A=15	
④	$A \leftarrow A-3$ 写回A=13

T1	T2
① 读A=50 读B=100 求和=150	
②	读B=100 $B \leftarrow B*2$ 写回B=200
③ 读A=50 读B=200 求和=250 (验算不对)	

T1	T2
① 读C=100 $C \leftarrow C*2$ 写回C	
②	读C=200
③ ROLLBACK C恢复为100	导致T2读出的数据和数据库中的数据不一致

2. 封锁 < 排他锁 (X锁、写锁) — 一个X锁上不能加其他任何锁
共享锁 (S锁、读锁) — 一个S锁上可以加X锁

3. 活锁: 某事务永远等待锁

预防: 采用FCFS策略

死锁: T1封锁R1又请求R2, T2封锁R2又请求R1
导致T1和T2僵死, 永远不能执行完毕。

预防: ① 一次封锁法 ② 顺序封锁法
要求事务一次性将所有要用到的数据加锁
所有事务按一定的顺序对数据实施封锁

诊断: ① 超时法 ② 等待图法

4. 并发调度的可串行性

· 串行的调度策略一定是正确的

· 一个并发调度的结果是正确当且仅当该结果与某一串行调度的结果一致

5. 两段锁协议 (2PL) — 单个事务内必须先不断加锁 (扩张), 再不断解锁 (收缩)
(若只有扩张无收缩也算符合2PL)

- 在对任何数据进行读、写前, 必须要申请并获得对该数据的封锁
- 在释放一个封锁后, 事务不再申请和获得任何其他封锁

⊗ 一次封锁遵守2PL, 但2PL并不要求事务一次性将所有用到的数据加锁, 因此遵守2PL的事务可能发生死锁。

注意:

1. 关系代数5个基本操作: $\cup, -, \cap, \pi, \sigma$, 4个组合操作: $\bowtie, \div, \rho, \infty$
2. 写不写 distinct 根据实际情况分析
3. 外码考的是码(不是主码). 若要考非码, 则应使该非码为 unique, not null.
4. 判断某关系最高能到几范式, 应先求出所有的码. 再用 2NF, 3NF, BCNF 的判定规则去分析.
5. 关系的 **外延** 指关系中所有元组值的集合, 也称为关系的 **值**. 是 **动态的**. 关系的 **内涵** 指关系模型, 是 **静态的**, 记为 $RCU(F)$.
6. 求码: $L, R, LR, N, X = LUN, Y = LR$. 只有当 $(XA)^+ = U$ 时, 才能更新 $Y = Y - A$.
7. 实体: 客观存在并可以相互区分的事物
实体型: 用实体名及其属性名的集合来刻画和抽象同类实体
实体集: 同类实体的集合
8. 若 select / group by / order by 后的属性出现在 ≥ 2 个表中, 则应标识该属性选的是哪个表的. 如:

```
select BR.Bno, Bname, avg(Interval)
from BR, Book
where BR.Bno = Book.Bno
group by BR.Bno, Bname;
```

 (即: 同名属性必须加表前缀)
9. group by 之前可用 where 先筛一遍符合条件的元组.
10. 判定范式要先求出所有候选码.